

Malware Analysis Portfolio

Malicious Office Document Investigations — LetsDefend Blue Team Labs

Analyst	Asha Latha Panuganti
Programme	MSc Cyber Security, Aston University
Date	30 September 2026
Cases	1. Malicious Doc (CVE-2017-11882 RTF exploit) 2. Malicious VBA (obfuscated macro downloader)

Skills demonstrated

Malware triage	Static analysis	VirusTotal pivoting	Sandbox report reading	VBA deobfuscation	CyberChef	IOC extraction
MITRE ATT&CK mapping	Detection engineering					

At a glance

Case	Delivery	Technique	Outcome
Malicious Doc	Zipped RTF disguised as <code>factura.doc</code>	Equation Editor stack overflow (CVE-2017-11882)	Downloads <code>jan2.exe</code> , runs as <code>aro.exe</code>
Malicious VBA	Macro-enabled <code>inf.docm</code> invoice	Hex-obfuscated macro, HTTP download, WMI execution	Drops <code>dropped.exe</code> in <code>%TEMP%</code> and runs it via WMI

Case 1 — Malicious Doc

RTF

CVE-2017-11882

Downloader

Scenario

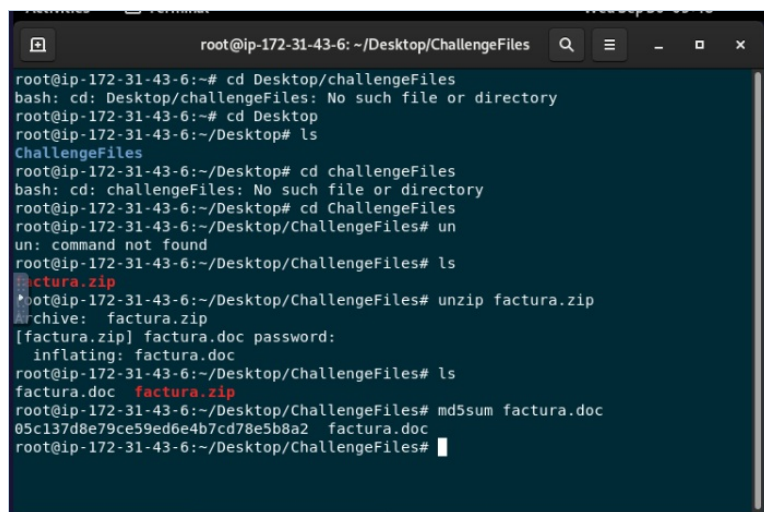
A password-protected archive, `factura.zip` ("factura" is Spanish for "invoice"), was provided for analysis. Goal: identify the exploit, CVE, downloaded payload, C2 address and the file written to disk.

Results

#	Question	Answer	Evidence
1	Exploit type	Stack-based buffer overflow	CVE research
2	CVE	CVE-2017-11882	Fig 1.3, 1.4
3	Malware downloaded	<code>jan2.exe</code>	Fig 1.5, 1.6
4	IP and port	<code>185.36.74.48:80</code>	Behavior → IP Traffic
5	Exe dropped to disk	<code>aro.exe</code>	Fig 1.7

Step 1 — Extraction and hashing

The archive was extracted in the isolated lab VM and the document hashed for reputation lookup.



```
root@ip-172-31-43-6: ~/Desktop/ChallengeFiles
root@ip-172-31-43-6:~# cd Desktop/challengeFiles
bash: cd: Desktop/challengeFiles: No such file or directory
root@ip-172-31-43-6:~# cd Desktop
root@ip-172-31-43-6:~/Desktop# ls
ChallengeFiles
root@ip-172-31-43-6:~/Desktop# cd challengeFiles
bash: cd: challengeFiles: No such file or directory
root@ip-172-31-43-6:~/Desktop# cd ChallengeFiles
root@ip-172-31-43-6:~/Desktop/ChallengeFiles# un
un: command not found
root@ip-172-31-43-6:~/Desktop/ChallengeFiles# ls
factura.zip
root@ip-172-31-43-6:~/Desktop/ChallengeFiles# unzip factura.zip
Archive:  factura.zip
[factura.zip] factura.doc password:
  inflating: factura.doc
root@ip-172-31-43-6:~/Desktop/ChallengeFiles# ls
factura.doc  factura.zip
root@ip-172-31-43-6:~/Desktop/ChallengeFiles# md5sum factura.doc
05c137d8e79ce59ed6e4b7cd78e5b8a2  factura.doc
root@ip-172-31-43-6:~/Desktop/ChallengeFiles#
```

Fig 1.1 — Extracting `factura.doc` (password `infected`) and calculating its MD5.

File	Hash
factura.doc (MD5)	<code>05c137d8e79ce59ed6e4b7cd78e5b8a2</code>
factura.doc (SHA-256)	<code>5a31c77293af2920d7020d5d0236691adcea2c57c2716658ce118a5cba9d4913</code>
factura.zip (SHA-256)	<code>df9f9bed6685eeba38defa87a755a0c8120f7b572094368e3aef41829039980d</code>

Step 2 — Pivot from the archive to the document

The ZIP's VirusTotal report only described the container, and its contacted IPs were Microsoft/Google background traffic. The real sample was reached through **Relations** → **Bundled Files**, which also revealed the true file type: **Rich Text Format**, not a Word binary.

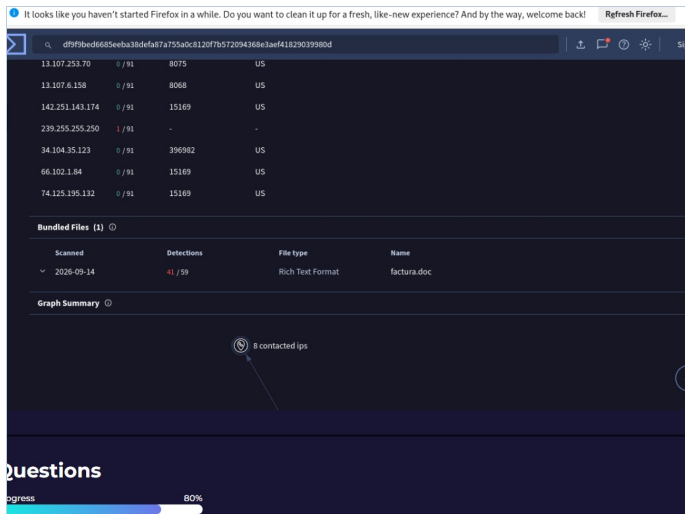


Fig 1.2 — Bundled file inside the ZIP: `factura.doc`, Rich Text Format, 41/59 detections.

Q1 & Q2 — Exploit and CVE

The report tags list `cve-2017-11882`, `exploit` and `ole-control`. Two CVEs appear in the tags, so vendor labels were compared: most name CVE-2017-11882 (Avast, AVG, ClamAV, Fortinet, Huorong, McAfee, Symantec, Tencent, Trellix, Varist); only WithSecure names CVE-2017-0199.

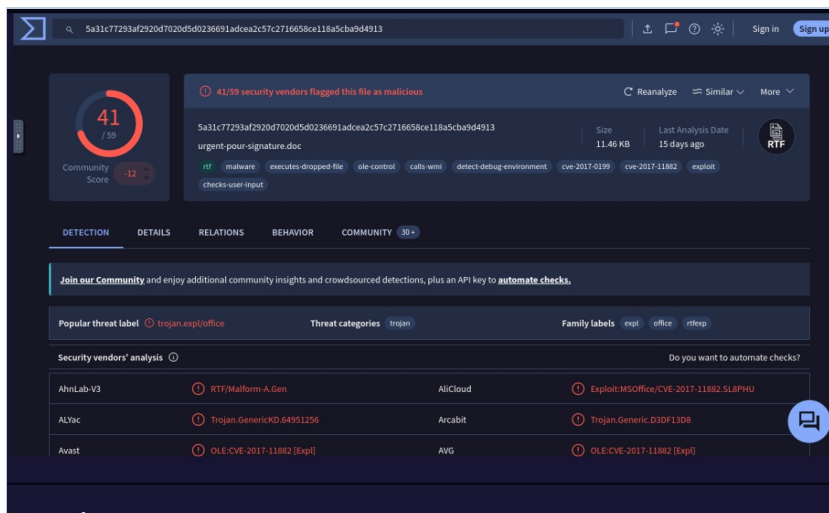


Fig 1.3 — Document report: 41/59 detections, RTF type, CVE tags.

It looks like you haven't started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back!

Refresh Firefox...

5a31c77293af2920d7020d5d0236691adcea2c57c2716658ce118a5c8a9d4913

Security vendors' analysis

Do you want to automate checks?

AhnLab-V3	RTF/Malform-A.Gen	AliCloud	Exploit:MSOffice/CVE-2017-11882.SLBPHU
ALYac	Trojan.GenericKD.64951256	Arcabit	Trojan.Generic.D3DF1308
Avast	OLE/CVE-2017-11882 [ExpI]	AVG	OLE/CVE-2017-11882 [ExpI]
Avira (no cloud)	HEUR/RTF.Malformed	BitDefender	Trojan.GenericKD.64951256
ClamAV	RTf.Exploit.CVE_2017_11882-6584355-1	CTX	RTf.exploit-kit.generic
Cynet	Malicious (score: 99)	DrWeb	Exploit.ShellCode.69
Emsisoft	Trojan.GenericKD.64951256 (B)	eScan	Trojan.GenericKD.64951256
ESET-NOD32	Generic.KQIKNIC.Trojan	Fortinet	MSOffice/CVE_2017_11882.Alexploit
GData	Trojan.GenericKD.64951256	Google	Detected
Huorong	Exploit/CVE-2017-11882_gen	KTAntiVirus	Trojan (0057b3a91)
K7GW	Trojan (0057b3a91)	Kaspersky	UDS.Exploit.MSOffice.Generic
Lionic	Trojan.MSOffice.Generic.31c	McAfee Scanner	Trojan/Office/CVE201711882.A

Fig 1.4a — Vendor labels naming CVE-2017-11882.

It looks like you haven't started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back!

Refresh Firefox...

5a31c77293af2920d7020d5d0236691adcea2c57c2716658ce118a5c8a9d4913

Symantec	Exp.CVE-2017-11882/g2	TACHYON	Trojan-Exploit/RTF/CVE-2017-11882
Tencent	Exp.Office.CVE-2017-11882_a	Trellix ENS	Exploit-CVE2017-11882.yx
TrendMicro	TROJ_FRS.0NA103C121	TrendMicro-HouseCall	TROJ_FRS.0NA103C121
Varist	CVE-2017-11882.D.gen/Camelot	VIPRE	Trojan.GenericKD.64951256
VirtT	W97M.Dwldr.GKN	WithSecure	Exploit:W97M/CVE-2017-0199.B
Xcltium	Malware@F3v7jv@t6uph6	ZoneAlarm by Check Point	Troj/RTfExp-EQ
Zoner	Probably Heur.RTF.BadVersion	Acronis (Static ML)	Undetected
Antiy-AVL	Undetected	Bkav Pro	Undetected
CMC	Undetected	CrowdStrike Falcon	Undetected
Gridinsoft (no cloud)	Undetected	Jiangmin	Undetected
Kingsoft	Undetected	Malwarebytes	Undetected
MaxSecure	Undetected	Panda	Undetected
Undetected	Undetected	Undetected	Undetected

Fig 1.4b — Further labels; WithSecure is the only vendor citing CVE-2017-0199.

Exploit type: VirusTotal identifies the CVE, not the bug class, so the CVE was researched. EQNEDT32.EXE copies a font name from the embedded equation object into a fixed-size local buffer without a length check. An oversized name overwrites the saved return address on the stack, redirecting execution to shellcode — a **stack-based buffer overflow**.

Q3 — Downloaded malware

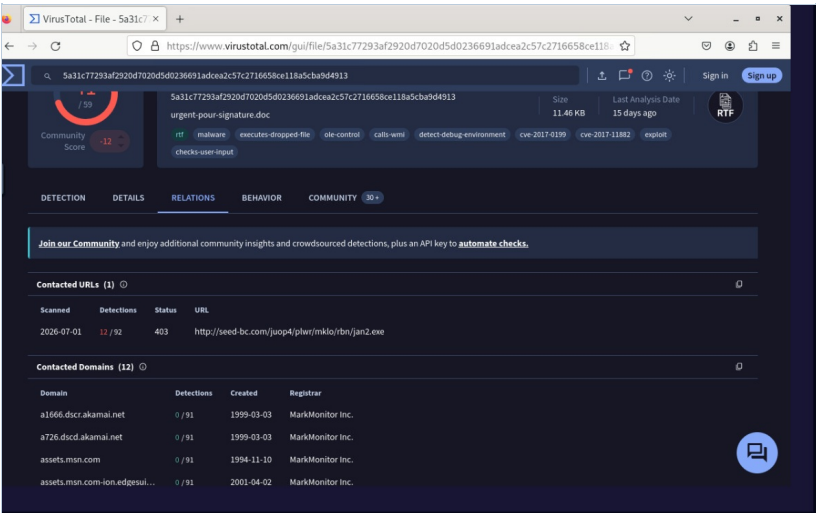


Fig 1.5 — Relations → Contacted URLs: request for jan2.exe .

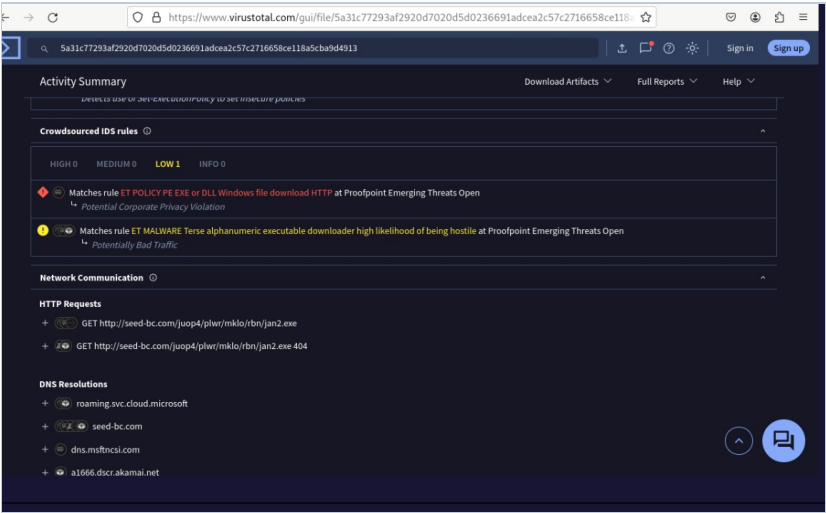


Fig 1.6 — Behavior: HTTP GET for jan2.exe and Proofpoint ET rules for an executable download.

Payload: jan2.exe from hxxp://seed-bc[.]com/juop4/plwr/mklo/rbn/jan2.exe

Q4 — IP and port

Behavior → Network Communication → IP Traffic shows the payload host at 185.36.74[.]148:80 (plain HTTP). Other addresses belong to Microsoft, Akamai and Google sandbox noise.

Q5 — File dropped to disk

The process list shows the chain: WINWORD.EXE opens the document, EQNEDT32.EXE -Embedding starts (the exploited component), then %APPDATA%\aro.exe runs. The payload is downloaded as jan2.exe but saved under a different name.

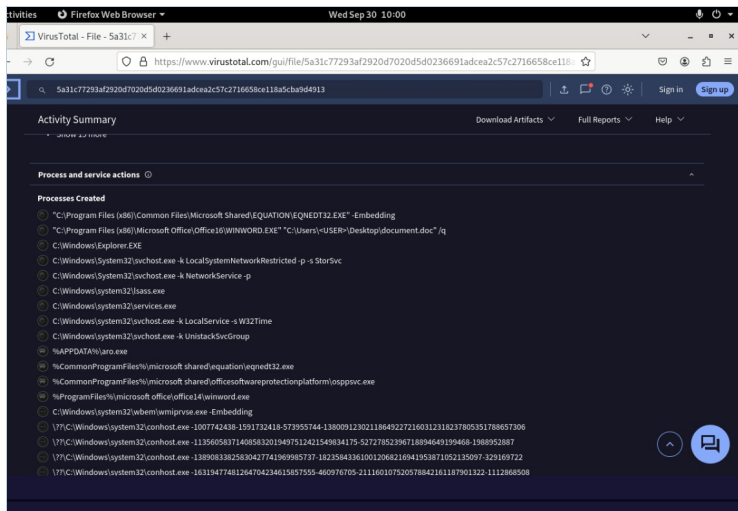


Fig 1.7 — Processes Created: EQNEDT32.EXE followed by %APPDATA%\aro.exe .

Attack chain

```
Invoice lure → factura.zip → factura.doc (RTF)
├─ WINWORD.EXE opens document
│   └─ EQNEDT32.EXE loads embedded Equation object
│       └─ Stack overflow (CVE-2017-11882) → shellcode
│           └─ HTTP GET jan2.exe from 185.36.74.48:80
│               └─ Saved and run as %APPDATA%\aro.exe
```

Case 2 — Malicious VBA

DOCM VBA macro Hex obfuscation WMI

Scenario

An employee received an invoice email with a macro-enabled document (`inf.docm`). The extracted macro (`invoice.vb`) was reviewed in a text editor, not executed, and its obfuscated strings decoded with CyberChef.

Results

Question	Answer	Evidence
Website hosting payload	<code>tinyurl.com</code> (<code>hxps://tinyurl[.]com/g2z2gh6f</code>)	Fig 2.2
Payload file name	<code>dropped.exe</code> (in <code>%TEMP%</code>)	Fig 2.3, 2.4
Connection object	<code>MSXML2.ServerXMLHTTP.6.0</code>	Fig 2.5, 2.6
User-Agent	<code>Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)</code>	Fig 2.7, 2.8
File read/write object	<code>ADODB.Stream</code>	Fig 2.9, 2.10
WMI execution object	<code>winmgmts:\\.\root\cimv2:Win32_Process</code>	Fig 2.11

Step 1 — Understanding the macro

Every sensitive string is hex-encoded and passed through `hgmneqolwgxg()`, the macro's hex-decode function. Tracing variables shows the logic: build a URL and a file path, download with an HTTP object, write the response to disk with a stream object, then call `jausltewrjghdtvi` to execute the file.

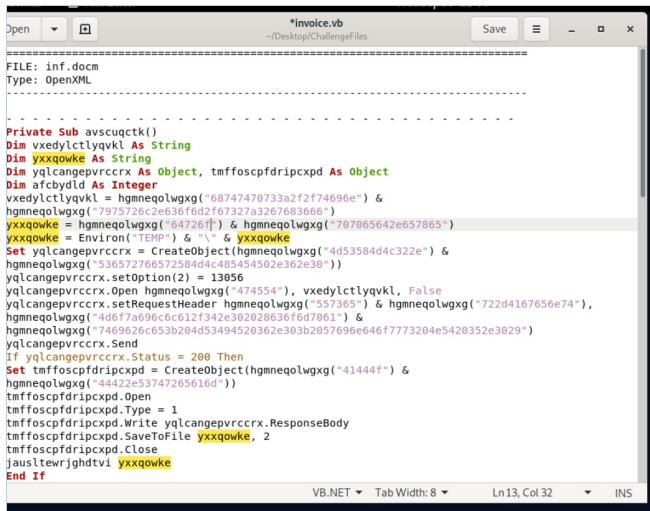


Fig 2.1 — Obfuscated macro; the payload-path variable `yxxqowke` traced from definition to `SaveToFile` and execution.

Payload host

`vxedytctlyqvkl` joins two hex strings; decoded and concatenated they form a URL-shortener link, which hides the real payload server.

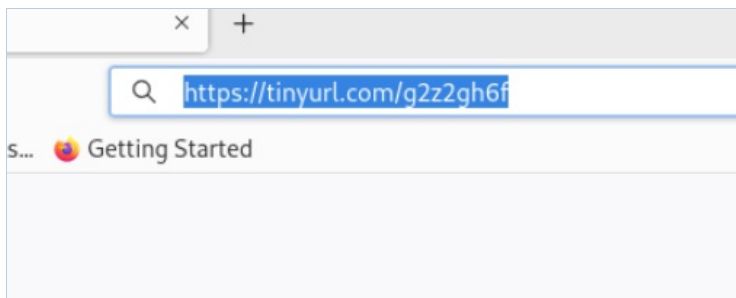


Fig 2.2 — Decoded download URL: `tinyurl.com/g2z2gh6f`.

User-Agent

The value passed to `setRequestHeader "User-Agent"` impersonates Internet Explorer 6 on Windows 2000 — over 20 years old, so it is a strong anomaly in modern proxy logs.

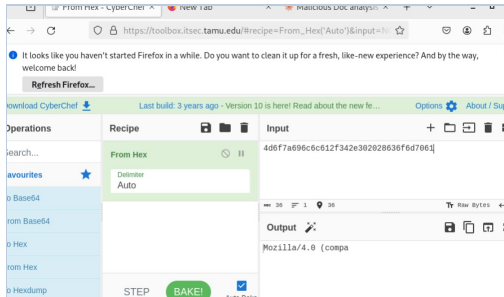


Fig 2.7 — First half of the User-Agent decoded.



Fig 2.8 — Full User-Agent string after joining both parts.

File read/write object

The second `CreateObject` is opened, set to binary (`Type = 1`), filled with `ResponseBody` and saved with `SaveToFile`.

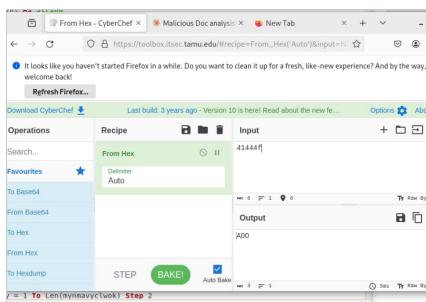


Fig 2.9 — 41444f → "ADO".

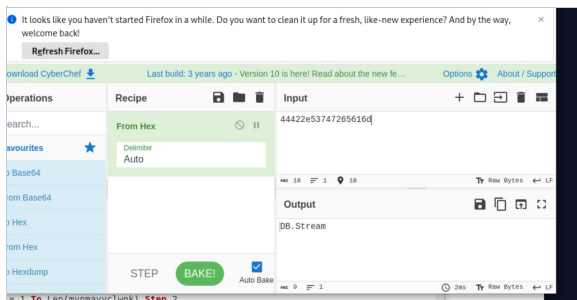


Fig 2.10 — 44422e53747265616d → "DB.Stream". Joined: ADODB.Stream.

WMI execution object

The execution function uses `GetObject` on a WMI path so the payload is launched by `WmiPrvSE.exe` rather than directly by Word, breaking the obvious parent-child chain that detections look for.

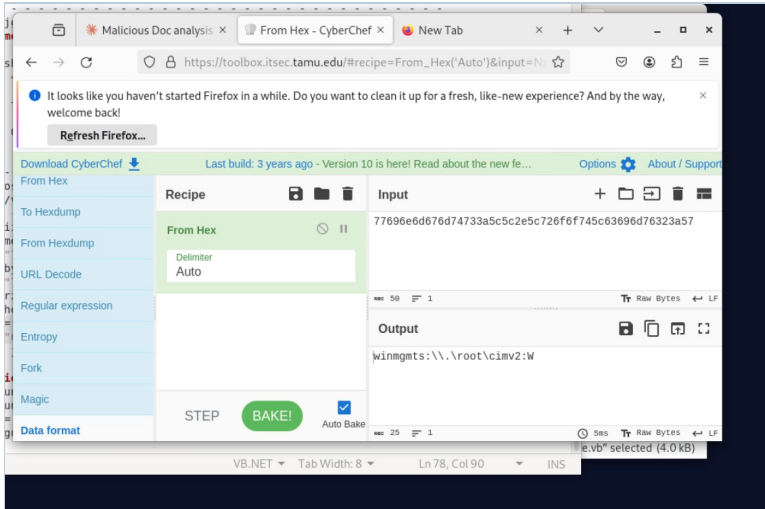


Fig 2.11 — Decoding the WMI object path (first segment shown); full value `winmgmts:\\.\\root\\cimv2:Win32_Process`.

Behaviour flow

```
User enables macros in inf.docm
├─ MSXML2.ServerXMLHTTP.6.0 GET hxtps://tinyurl[.]com/g2z2gh6f
  (User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0))
├─ HTTP 200 → ADODB.Stream writes body to %TEMP%\dropped.exe
  └─ WMI Win32_Process.Create launches dropped.exe (hidden window)
```

Consolidated Intelligence

MITRE ATT&CK mapping

Technique	Case 1	Case 2
T1566.001 Spearphishing Attachment	✓ invoice ZIP	✓ invoice DOCM
T1204.002 User Execution: Malicious File	✓	✓ enable macros
T1203 Exploitation for Client Execution	✓ CVE-2017-11882	
T1059.005 Command & Scripting: Visual Basic		✓ VBA macro
T1027 Obfuscated Files or Information		✓ hex strings
T1105 Ingress Tool Transfer	✓ jan2.exe	✓ dropped.exe
T1047 Windows Management Instrumentation		✓ Win32_Process
T1036 Masquerading	✓ RTF as .doc, renamed payload	✓ fake IE6 User-Agent

Indicators of compromise (defanged)

Type	Value	Case
SHA-256	5a31c77293af2920d7020d5d0236691adcea2c57c2716658ce118a5cba9d4913	1
MD5	05c137d8e79ce59ed6e4b7cd78e5b8a2	1
URL	hxxp://seed-bc[.]com/juop4/plwr/mklo/rbn/jan2.exe	1
IP:Port	185.36.74[.]48:80	1
File	%APPDATA%\aro.exe	1
URL	hxxps://tinyurl[.]com/g2z2gh6f	2
File	%TEMP%\dropped.exe	2
User-Agent	Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.0)	2

Detection and response recommendations

- **Patch and harden Office:** apply the Nov 2017 update for CVE-2017-11882 (Equation Editor was later removed entirely); block macros in files from the internet via Group Policy.
- **EDR hunt:** alert on `EQNEDT32.EXE` running at all, and on Office processes followed by executables launched from `%APPDATA%` or `%TEMP%`.
- **WMI:** alert on `WmiPrivSE.exe` spawning unsigned executables from user-writable folders.
- **Proxy:** flag legacy User-Agents such as MSIE 6.0, executable downloads over plain HTTP, and URL-shortener links resolving to `.exe` files.
- **Email gateway:** quarantine password-protected archives and macro-enabled documents from external senders.
- **Scope:** search proxy and DNS logs for every IOC above to find other affected hosts; block at firewall, proxy and EDR.

Lessons learned

- Verify the real file type — `factura.doc` was RTF.
- Analyse the inner sample, not the archive; the ZIP report showed only background traffic.
- VirusTotal tells you *which* CVE; the bug class comes from the vulnerability's technical documentation.
- When a CVE conflict appears, weigh vendor consensus against observed behaviour (`EQNEDT32.EXE` in the process tree).
- Macro obfuscation is often a single decode routine — find it once, then decode every string systematically and trace each variable to its use.
- Downloaded and saved file names can differ; confirm with the process tree or the code path.

Educational lab analysis performed in an isolated LetsDefend environment. Samples were not executed on personal systems; all network indicators are defanged.